

Programmering med mikrokontroller

Aut2602 Innlevering 1

Jens Christian Valen Leynse

September 2024

1 Introduksjon

I denne oppgaven skal IO-kortet kobles til for å teste og kontrollere lysdioder og knapper. Oppgaven innebærer å identifisere hvor portene til knapper og lysdioder er tilkoblet.

Kode segmentene i denne rapporten er delt inn i fem deler pluss felles segmenter, hver med sin egen detaljerte beskrivelse av kodens logikk og oppbygging for hver deloppgave.

2 Kode segmenter

Dette segmentet inkluderer nødvendige biblioteker og definerer konstanter og pinnekonfigurasjoner for lysdiodene og knappene. F_CPU defineres for å spesifisere klokkehastigheten, som er viktig for tidsforsinkelser i `_delay_ms`. Antall lysdioder og knapper er også listet, slik at konstantene som vil bli brukt senere er mer forståelige.

```
1 #include <avr/io.h>
2 #include <util/delay.h>
3 #include <avr/interrupt.h>
4
5 #define F_CPU 4000000UL
6 #define NUM_LEDS 4
7 #define NUM_BUTTONS 3
8
9 // Define pins for LEDs on PORTC
10 #define LED0_PIN 0 // PC0
11 #define LED1_PIN 1 // PC1
12 #define LED2_PIN 2 // PC2
13 #define LED3_PIN 3 // PC3
14
15 // Define pins for buttons on PORTD
16 #define BUTTON_SW1_PIN 0 // PD0
```

```

17 #define BUTTON_SW2_PIN 1 // PD1
18 #define BUTTON_SW3_PIN 2 // PD2
19
20 #define DEBOUNCE_DELAY 50
21 #define LOOP_DELAY 500

```

Denne funksjonen konfigurerer nødvendige porter for å styre LEDer og lese knapper. LED-pinnene settes som utganger og slås av ved start for å sikre at de starter i en kjent tilstand. Knappene konfigureres som innganger med aktiverte pull-up motstander, noe som sikrer at de leser høyt når de ikke er trykket ned. Knappene settes også til å registrere endringer på begge kanter (både stigende og fallende), for å håndtere både trykk og slipp av knappene.

```

1 void init_ports() {
2     // Set LED pins as outputs
3     PORTC.DIRSET = (1 << LED0_PIN) | (1 << LED1_PIN) |
4         (1 << LED2_PIN) | (1 << LED3_PIN);
5     // Turn them off initially
6     PORTC.OUTCLR = (1 << LED0_PIN) | (1 << LED1_PIN) |
7         (1 << LED2_PIN) | (1 << LED3_PIN);
8
9     // Set button pins as inputs
10    PORTD.DIRCLR = (1 << BUTTON_SW1_PIN) | (1 <<
11        BUTTON_SW2_PIN) | (1 << BUTTON_SW3_PIN);
12    // Enable pull-up and detect edges, read high when
13    // not pressed
14    PORTD.PIN0CTRL = PORT_PULLUPEN_bm |
15        PORT_ISC_BOTHEDGES_gc;
16    PORTD.PIN1CTRL = PORT_PULLUPEN_bm |
17        PORT_ISC_BOTHEDGES_gc;
18    PORTD.PIN2CTRL = PORT_PULLUPEN_bm |
19        PORT_ISC_BOTHEDGES_gc;
20 }

```

Denne funksjonen endrer tilstanden til parameter LED-pinnen.

```

1 void blink(uint8_t output) {
2     PORTC.OUTCLR = output; // Turn off LED
3     _delay_ms(LOOP_DELAY);
4     PORTC.OUTSET = output; // Turn on LED
5     _delay_ms(LOOP_DELAY);
6 }

```

a) Prøv å få lysdiodene merket D0-3 på IO-kortet til å blinke på forskjellige måter

Denne funksjonen kontrollerer en gruppe lysdioder ved å manipulere PORTC direkte gjennom en maske. Masken brukes til å angi hvilke lysdioder som skal aktiveres eller deaktiveres i et avtagende mønster, basert på antall definerte lysdioder.

```
1 void blink_all_leds_variously(void) {
2     // Blink in a receding pattern
3     for (uint8_t i = NUM_LEDS; i > 0; i--) {
4         uint8_t mask = 0;
5
6         // Create a mask for the LEDs to be controlled
7         for (uint8_t j = 0; j < i; j++) {
8             mask |= (1 << (LED0_PIN + j));
9         }
10
11         blink(mask);
12     }
13 }
```

b) Få lysdiodene til å skru seg på en etter en, ved å bruke en for-løkke

Denne funksjonen kontrollerer sekvensiell blinking av hver LED på en mikrokontroller. For hver LED i løkken, slår funksjonen først av LEDen, venter en definert periode, og slår deretter på LEDen igjen. Dette resulterer i at hver LED blinker én etter én.

```
1 void sequentially_activate_leds(void) {
2     for (uint8_t i = 0; i < NUM_LEDS; i++) {
3         blink((1 << i));
4     }
5 }
```

c) Få lysdiodene D0-2 til å lyse når knappene SW1-3 holdes inne. Husk at disse også er utsatt for prell. D3 kan settes til å blinke

Denne funksjonen kontrollerer lysdiodene D0-2 basert på tilstanden til knappene SW1-3, med enkel prellkontroll for å sikre pålitelig deteksjon av knappetrykk. Funksjonen sjekker kontinuerlig hver knapp, og hvis en knapp er trykket, tennes den tilsvarende LEDen. Hvis knappen ikke er trykket, slukkes LEDen. LED D3 blinker uavhengig for å vise at systemet er aktivt.

```
1 // Button is pressed if the bit is 0
2 uint8_t is_button_pressed(uint8_t button_pin) {
3     return !(PORTD.IN & (1 << button_pin));
4 }
5
6 void light_leds_on_button_press(void) {
7     // Assume all buttons are not pressed initially
8     uint8_t last_button_states = 0x07;
9
10    // Check each button and control corresponding LED
11    for (uint8_t i = 0; i < NUM_BUTTONS; i++) {
12        if (is_button_pressed(i)) {
13            PORTC.OUTCLR = (1 << (i - BUTTON_SW1_PIN))
14                ; // Turn on corresponding LED
15        } else {
16            PORTC.OUTSET = (1 << (i - BUTTON_SW1_PIN))
17                ; // Turn off corresponding LED
18        }
19    }
20
21    // Blink LED3 only
22    blink(1 << LED3_PIN);
23 }
```

d) Få lysdiodene D0-2 til å skifte tilstand hver gang knappen trykkes inn (Toggle)

Denne funksjonen håndterer logikken for å kontrollere LEDer basert på tilstanden til knappene. Den sjekker kontinuerlig tilstanden til hver knapp ved hjelp av tilstands hjelpefunksjonen. Hvis en knapp trykkes (detektert ved en endring fra høy til lav), toggles tilstanden til den tilsvarende LEDen ved hjelp av den andre hjelpefunksjonen, som endrer tilstanden til LEDen. Dette oppnås ved å bruke en tilstandsvariabel for LEDene, som oppdateres basert på knappetrykk. Debouncing implementeres for å håndtere prell, og en liten forsinkelse

reduseres CPU-bruken. Funksjonen er designet for å være modulær, siden neste oppgave "interrupts" gjentar mye av den samme logikken.

```
1 uint8_t read_button_state(uint8_t button_pin) {
2     return !(PORTD.IN & (1 << button_pin));
3 }
4
5 void set_led_state(uint8_t led_pin, uint8_t state) {
6     if (state) { // Turn on LED
7         PORTC.OUTCLR = (1 << led_pin);
8     } else { // Turn off LED
9         PORTC.OUTSET = (1 << led_pin);
10    }
11 }
12
13 void toggle_leds_on_button_toggle(void) {
14     // Ensure all LEDs are initially off
15     uint8_t led_states = 0;
16     // Buttons are not pressed initially (HIGH)
17     uint8_t last_button_states = 0x07;
18
19     for (uint8_t i = 0; i < NUM_BUTTONS; i++) {
20         uint8_t current_button_state =
21             read_button_state(i);
22
23         // Toggle the corresponding LED state if the
24         // button is pressed
25         if (current_button_state == 1 && !(
26             last_button_states & (1 << i))) {
27             led_states ^= (1 << i);
28             set_led_state(i, led_states & (1 << i));
29             _delay_ms(DEBOUNCE_DELAY);
30         }
31
32         // Update the last button state
33         if (current_button_state) {
34             last_button_states |= (1 << i);
35         } else {
36             last_button_states &= ~(1 << i);
37         }
38     }
39     _delay_ms(10); // Small delay to reduce CPU usage
40 }
```

e) Repeter samme funksjonalitet som forrige punkt, men bruk programavbrudd (interrupt)

ISR (Interrupt Service Routine) håndterer debouncing og sjekker hver knapp for å toggle den tilsvarende LEDen kun på stigende signal. Dette sikrer at LEDene endrer tilstand basert på faktiske brukertrykk. ISR bruker statiske volatile variabler for å spore tilstanden til LEDene og knappene, noe som er nødvendig siden disse variablene oppdateres i ISR og for å unngå gjentakende navnekonflikter. ISR sjekker hver knapp og toggler tilstanden til den tilsvarende LEDen basert på knappens nåværende og forrige tilstand, og implementerer en enkel debouncing-logikk for å sikre pålitelig deteksjon av knappetrykk.

```
1 ISR(PORTD_PORT_vect) {
2     _delay_ms(DEBOUNCE_DELAY);
3     static volatile uint8_t led_states = 0;
4     static volatile uint8_t last_button_states = 0x07;
5
6     // Check each button and toggle the corresponding
7     // LED only on rising edge
8     for (uint8_t i = 0; i < 3; i++) {
9         uint8_t current_button_state =
10             read_button_state(i);
11
12         // Check for rising edge
13         if (current_button_state == 1 && !(
14             last_button_states & (1 << i))) {
15             led_states ^= (1 << i); // Toggle LED
16             // state
17             set_led_state(i, led_states & (1 << i));
18         }
19
20         // Update the last button state
21         if (current_button_state) {
22             last_button_states |= (1 << i);
23         } else {
24             last_button_states &= ~(1 << i);
25         }
26     }
27
28     // Clear interrupt flags
29     PORTD.INTFLAGS = PORT_INT0_bm | PORT_INT1_bm |
30                     PORT_INT2_bm;
31 }
```

Hovedfunksjonen i dette programmet initialiserer systemportene og aktiverer globale avbrudd. Deretter, innenfor en evig løkke, definerer den en switch-case-struktur som bestemmer hvilken funksjonalitet som skal utføres.

```
1 // Main function with switch-case for different
  options
2 int main(void) {
3     init_ports();
4     sei(); // Enable global interrupts
5
6     // Set the option based on your use case (1-5)
7     uint8_t option = 1;
8
9     while (1) {
10         switch (option) {
11             case 1:
12                 blink_all_leds_variously();
13                 break;
14             case 2:
15                 sequentially_activate_leds();
16                 break;
17             case 3:
18                 light_leds_on_button_press();
19                 break;
20             case 4:
21                 toggle_leds_on_button_toggle();
22                 break;
23             case 5:
24                 // Option 5 relies on interrupts, so
25                 // nothing else to do here
26                 break;
27             default:
28                 // Handle invalid option
29                 break;
30         }
31     }
32     return 0;
}
```

3 Gjennomføring

Oppgavene ble først utviklet separat for å sikre at hver del fungerte som forventet. Senere ble disse integrert i en felles fil for å forenkle strukturen og redusere gjentakende kode. Ved å bruke hjelpefunksjoner og kombinere logikk der det var mulig, ble koden mer effektiv. I den endelige versjonen av prosjektet ble hver oppgave plassert i sin egen funksjon, og disse ble aktivert gjennom en switch-case i main-funksjonen, noe som gjorde koden ryddig og lett å følge.

4 Resultat

Prosjektet oppnådde de ønskede resultatene. Systemet fungerte som forventet under testing, med hver LED og knapp som reagerte korrekt i henhold til de definerte kravene.

5 Diskusjon

Under prosjektet oppsto en interessant observasjon relatert til bruken av OUTCLR og OUTSET for styring av LEDene. Opprinnelig var forventningen at for å oppnå en blinkende effekt, måtte LEDene først slås på og deretter slås av ved å bruke OUTCLR. Imidlertid, gjennom testing og eksperimentering, ble det klart at å starte med OUTCLR (som slår av LEDene) og deretter bruke OUTSET (som slår dem på igjen) faktisk produserte den ønskede blinkende effekten til oppgavene.